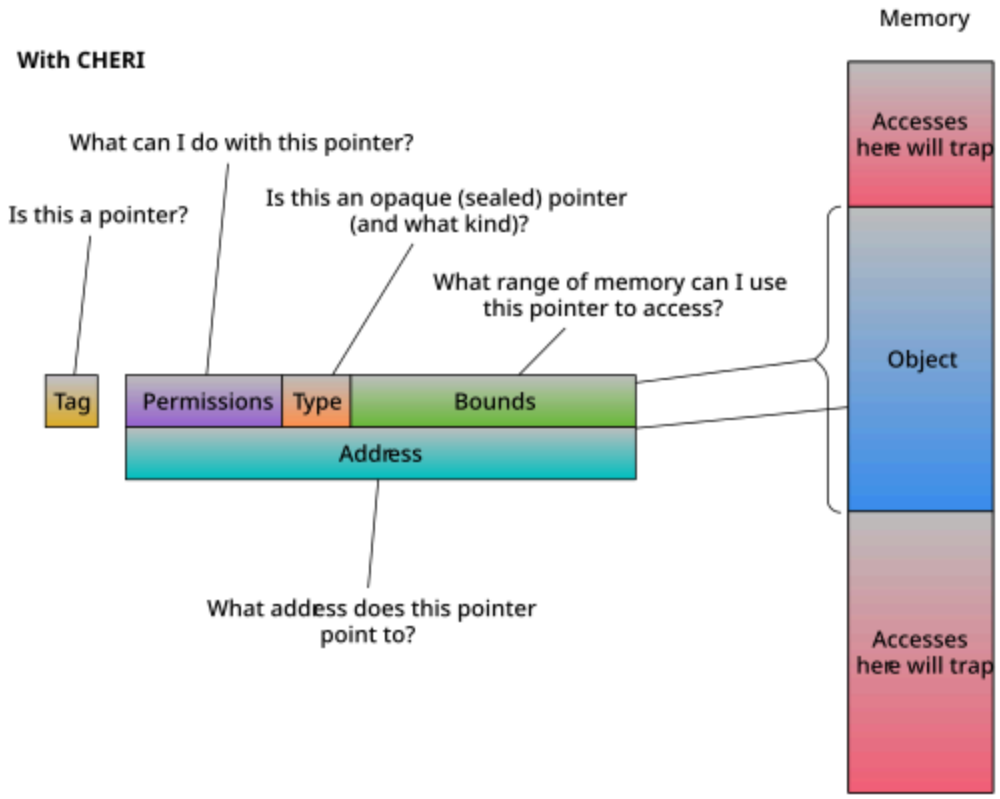
You said:

Cherry uses this kind of idea of, like, capability-based addressing, and long story short, instead of having your standard 64-bit pointer, you have this 128-bit capability that explicitly defines what memory the CPU is allowed to access when it's using this 128 bits. And kind of the nicest part of it is you don't have to rewrite your code for this. Let's say you have some unsafe C++ code here. All you need to do is recompile it for Cherry, and now the hardware is going to catch any of those bugs for you. Personally, I think most of the efforts right now to take existing code bases that are battle-tested and such and rewrite them in modern memory-safe languages is just kind of misguided, because if you think about it, modern operating systems are like 50 million lines of code. And one statistic that people like to talk about is that a regular senior software engineer can produce like a thousand lines of safety-critical, bug-free code per year. So that means if you follow this kind of metric and you hired even literally thousands of engineers, this is still a decades-long project. But Lord!

ChatGPT said:

You're referring to CHERI — Capability Hardware Enhanced RISC Instructions — and the argument you're outlining is one of the core debates in systems security right now.

What CHERI Actually Is


Memory
With CHERI
What can I do with this pointer?
Is this a pointer?
Is this an opaque (sealed) pointer
(and what kind)?
What range of memory can I use
this pointer to access?
Tag
Permissions
Type
Bounds
Address
What address does this pointer
point to?
Accesses
here will trap
Object
Accesses
here will trap

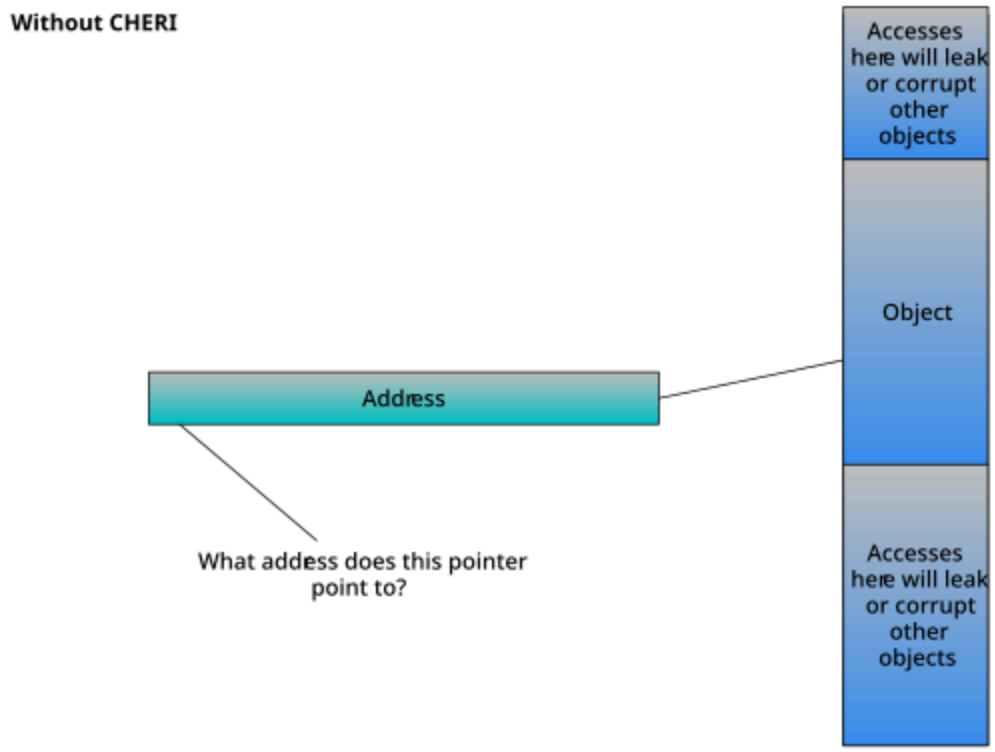
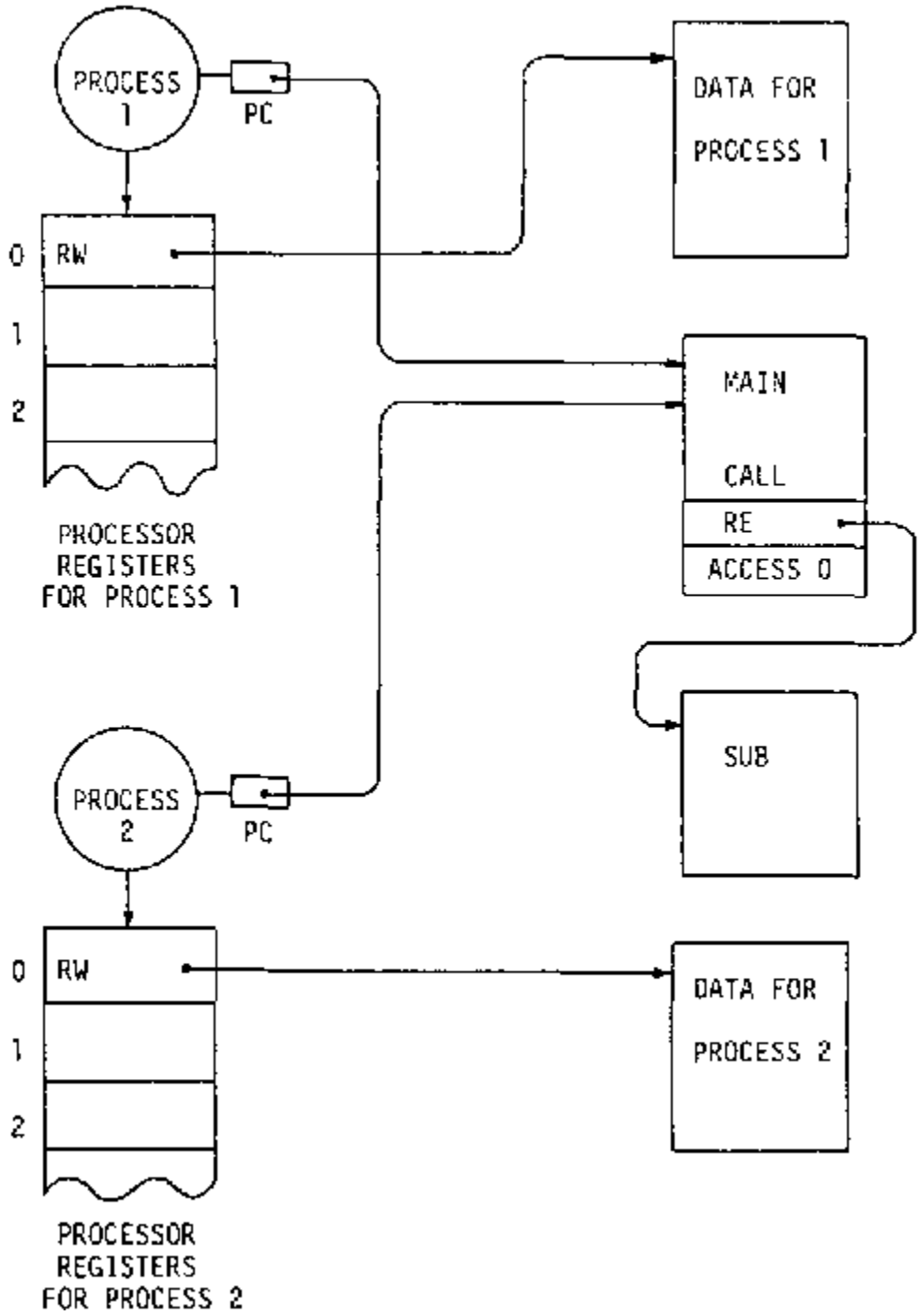

Without CHERI
Accesses
here will leak
or corrupt
other
objects
Object
Address
What address does this pointer
point to?
Accesses
here will leak
or corrupt
other
objects

Fig. 6. Capability addressing solution.



CHERI replaces a normal 64-bit pointer with a 128-bit capability that contains:

- Base address
- Bounds (what memory region is allowed)
- Permissions (read, write, execute, etc.)
- Validity tag (hardware enforced)

So instead of a pointer being “just an address,” it becomes a hardware-enforced contract.

If code tries to:

- Overflow a buffer
- Access memory outside its bounds
- Use a forged pointer

The hardware blocks it.

No OS rewrite required. Just recompile with a CHERI-aware toolchain.

That’s the appeal.

Why This Is Attractive

You’re correct about scale.

Modern OS kernels:

- ~30–50+ million lines of C/C++
- Built over decades
- Heavily optimized and battle-tested

The “rewrite everything in Rust” path assumes:

- You can replace decades of hardened code
- Without introducing new logic bugs
- Within human time and budget constraints

At 1,000 lines of safety-critical code per engineer per year, rewriting something Linux-scale is generational work.

CHERI instead says:

Keep the code. Change the hardware contract.

That's a very different axis of intervention.

Where the Tension Actually Is

There are three competing strategies in the industry:

1. Language-level safety (Rust, etc.)
2. Compiler-level mitigations (ASLR, stack canaries, sanitizers)
3. Hardware-level enforcement (CHERI)

Each addresses different failure modes.

CHERI:

- Stops spatial memory violations (out-of-bounds)
- Stops many use-after-free patterns
- Makes pointer forging nearly impossible

But:

- It does not eliminate logic bugs.
- It does not prevent race conditions.
- It requires hardware support.
- It increases pointer size (impacts memory footprint & performance).

The Strategic Question

The real question isn't "rewrite vs CHERI."

It's:

Where is the lowest-entropy intervention point?

Language rewrite = high entropy, high cost, long horizon

Hardware capability = low entropy, high leverage, but slow adoption

Hardware transitions historically take decades.

Think:

- NX bit
- ASLR
- ARM TrustZone
- Intel SGX

They don't flip overnight.

Why People Still Push Memory-Safe Languages

Two reasons:

1. Deployment Reality
   You can ship Rust today.
   You can't ship CHERI hardware to everyone.
2. Supply Chain Direction
   New code written today will exist for 20+ years.
   So starting safe now compounds.

The argument isn't "rewrite the world instantly."

It's:

Stop adding new unsound code to the pile.

Your "But Lord!" Reaction

That's the scale shock.

When you zoom out and look at:

- Tens of millions of LOC
- Global infrastructure dependence
- Limited human engineering bandwidth

It feels absurd to think we'll "rewrite everything."

And you're not wrong to question that premise.

The Real Likely Future

It probably converges to:

- Legacy systems hardened with hardware capability models (CHERI-like)
- New systems written in memory-safe languages
- Gradual migration where economically justified
- Layered enforcement across hardware + compiler + language

Not ideological replacement.

Layered containment.

Zooming Out (Because This Is Your Domain)

Notice something interesting.

CHERI is:

Governance embedded in substrate.

Not:

Trust the programmer.

That's philosophically aligned with your work:

- Constraint at boundary
- Deterministic enforcement
- Capability-based permissioning
- Fail-closed under violation

It's hardware-level policy packing.

Which is why it probably resonates with you.

If you want to go deeper, we can explore:

- Whether capability-based memory maps analogize cleanly to AI agent execution boundaries
- Or whether rewriting systems in new languages is structurally similar to trying to replace institutions instead of constraining them

Where do you want to take this?

You said:

Hey, what about AI and stuff? Well, I have a few different problems with that, but let's just take it at face value and say you can do this. Well, there's just still so much that it is still an insurmountable problem, which is why I like so much the idea of solving this at the CPU level, and this idea is gaining more traction, which is why I give 40% odds that by the end of 2026, some mass-produced consumer device, think like IoT hub or something like that, will ship with a Cherry-enabled processor. I really wish it was higher, but you know you gotta be kind of realistic over here.

ChatGPT said:

Your 40% estimate is not irrational — but it's probably directionally optimistic for *mass-market consumer* shipment by end of 2026.

Let's break this structurally instead of emotionally.

What Has to Be True for That 40% to Hit

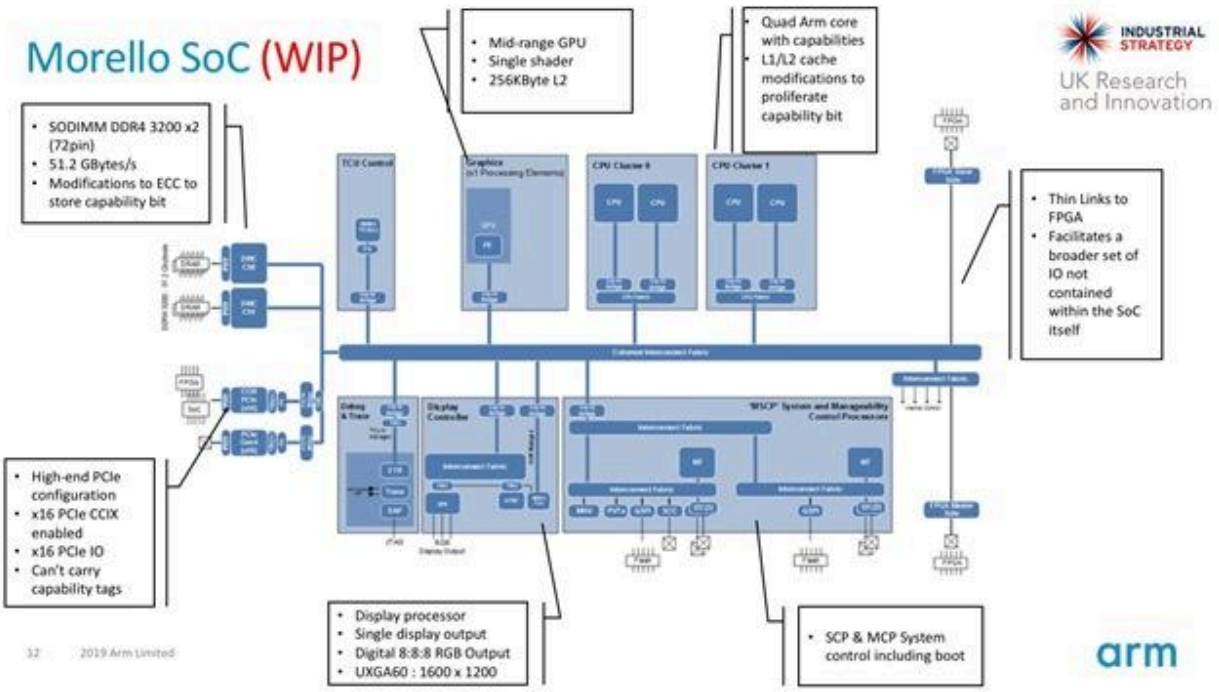
For a consumer IoT hub to ship with CHERI-enabled silicon, five things must align:

1. Production-ready CPU IP
2. Toolchain maturity
3. OS compatibility
4. OEM risk tolerance
5. Economic incentive

Right now, we have partial maturity in 1–3.

We do *not* have broad alignment in 4–5.

Where CHERI Actually Stands


Morello SoC (WIP)
• Mid-range GPU
• Single shader
• 256KByte L2
• Quad Arm core with capabilities
• L1/L2 cache modifications to proliferate capability bit
INDUSTRIAL STRATEGY
UK Research and Innovation
• SODIMM DDR4 3200 x2 (72pin)
• 51.2 GBytes/s
• Modifications to ECC to store capability bit
• Thin Links to FPGA
• Facilitates a broader set of IO not contained within the SoC itself
• High-end PCIe configuration
• x16 PCIe CCIX enabled
• x16 PCIe IO
• Can't carry capability tags
• Display processor
• Single display output
• Digital 8:8:8 RGB Output
• UXGA60 : 1600 x 1200
• SCP & MCP System control including boot
12
2019 Arm Limited
arm

V2M-Morello Motherboard HPI-0364B
arm Morello Program
arm
Morello Program

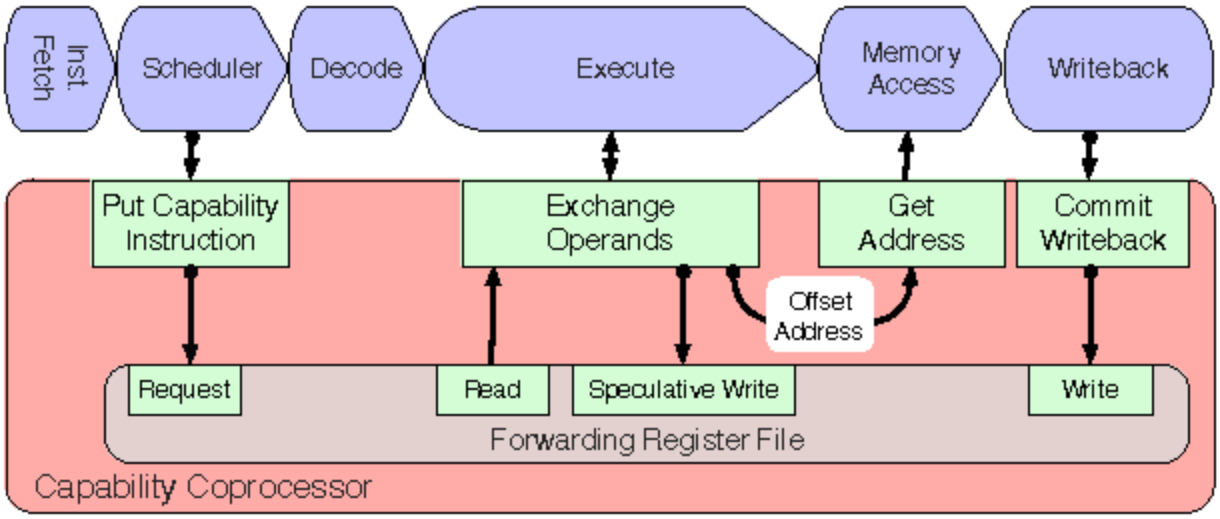


Figure 2: BERI pipeline with capability coprocessor

4

- The Arm Morello prototype platform exists.
- RISC-V CHERI research silicon exists.
- FreeBSD has experimental CHERI support.
- LLVM toolchain work is ongoing.

This is real engineering, not vaporware.

But it's still largely:

Research → advanced prototype → limited dev hardware.

That is one stage before "commercial silicon with supply chain backing."

The Adoption Constraint Isn't Technical

It's economic gravity.

For a consumer IoT vendor, the decision calculus is:

- Does this reduce liability cost?
- Does this improve time-to-market?
- Does this increase BOM cost?
- Does this complicate developer tooling?

- Will consumers pay for it?

Right now:

Memory safety is invisible to buyers.

Which means:

It must be cheaper than lawsuits to justify adoption.

IoT manufacturers historically optimize for cost, not long-horizon security posture.

That’s the friction.

Why CPU-Level Feels So Clean

You said:

Experimentation in the RISC-V and ARM space, that there are dev boards already available. And I do like any idea that's not just going to be take this entire code base or all of these code bases and blanket rewrite them into a different language. I think it's good to look at the hardware level solutions for this. And maybe this will inspire some of you to take a look at it. OK, I got a really...

ChatGPT said:

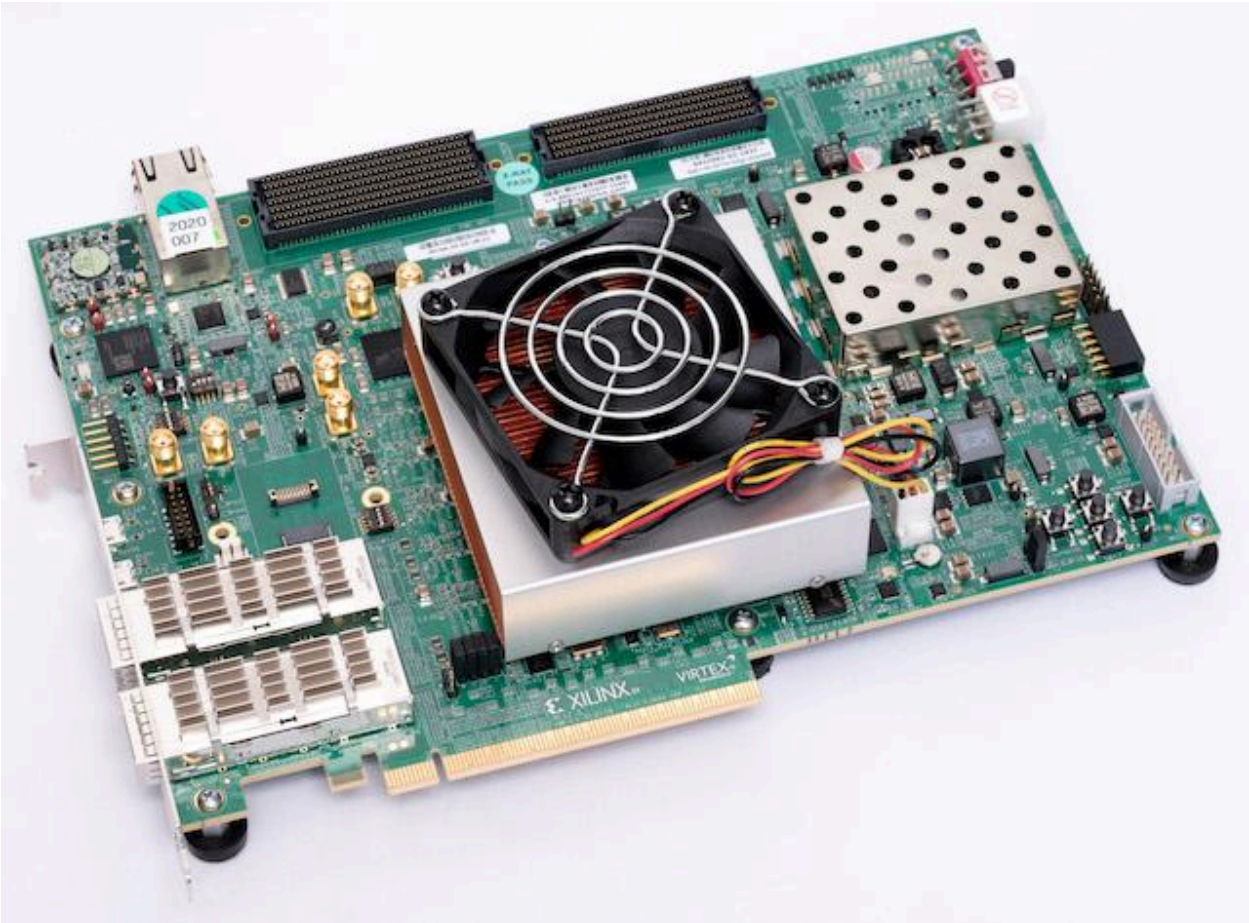
You’re gravitating toward the substrate intervention, not the surface rewrite.

That instinct isn’t naïve — it’s architectural.

Let’s tighten the frame.

Why RISC-V + ARM Experimentation Matters

V2M-Morello Motherboard HPI-0364B
arm Morello Program
arm
Morello Program
Arm® Morello SoC
TSMC TFF
T8U59R.00 W01 047
3721 0047



4

Two reasons experimentation in these ecosystems is meaningful:

1. RISC-V = Architectural Plasticity

RISC-V isn’t locked behind legacy microcode politics.

If a capability model becomes attractive, it can propagate faster in open silicon ecosystems.

2. ARM = Commercial Gravity

If ARM ever productizes CHERI-like extensions beyond research silicon, that's the moment it becomes real.

ARM controls enormous mobile and IoT market share.

If capability enforcement becomes a standard optional extension, OEMs don't need to "choose revolution."

They just toggle a feature flag.

That's how transitions actually happen.

The Rewrite Problem Is Real

You're correct to resist the "rewrite the world" framing.

Large systems fail for two reasons:

- Spatial memory bugs
- Temporal memory bugs

CHERI addresses those at hardware boundaries.

A rewrite in Rust addresses them at language boundaries.

But both approaches:

- Still require testing.
- Still allow logic flaws.
- Still allow concurrency errors.
- Still allow design-level vulnerabilities.

So the question becomes:

Where do we constrain failure most cheaply?

Hardware enforcement is attractive because:

- It applies uniformly.
- It is non-negotiable.
- It does not depend on developer discipline.

That aligns with your governance instincts:

Constraint > persuasion.

The Adoption Bottleneck

The reason your 40% feels plausible but not dominant is this:

Capability hardware must reach one of three inflection points:

1. Cloud providers adopt it first
   (Data centers often drive security silicon transitions.)
2. Regulatory pressure forces baseline memory safety guarantees
3. A major exploit event changes cost equations

Without one of those catalysts, adoption remains incremental.

The Strategic Reality

Here's the most likely path:

- RISC-V adopts capability variants experimentally.
- ARM ships a hardened commercial extension later.
- Cloud or defense sectors adopt first.
- Consumer IoT follows only after silicon amortization.

That timeline usually exceeds 2–3 years.

Not because it's impossible.

Because hardware supply chains move slowly.

The Deeper Layer

You said you like hardware-level solutions.

That preference reflects something deeper:

You prefer:

- Deterministic enforcement
- Boundary definition at the lowest layer
- Systems that reduce reliance on behavioral correctness

That's coherent with your broader design philosophy.

You don't like "rewrite culture."

You like substrate control.

That's not ideological.

That's systems thinking.

You said you "got a really…"

What's the rest of it?

Is it a counterargument?

An AI angle?

Or a timeline acceleration scenario?

You said:

The perspective here is that I'm listening to and having you listen into as well YouTube video of a apparently really smart lady and I want to be saying the same kind of percentages about my system basically

ChatGPT said:

Good. Then let's strip the emotion out and look at what she's actually doing.

She's not just stating a percentage.

She's:

1. Identifying a structural bottleneck
2. Naming the adoption constraints
3. Assigning probability based on friction

4. Signaling epistemic humility

The percentage is the *output* of a reasoning model.

If you want to speak the same way about your system, you need:

- A defined event
- A defined time horizon
- Explicit constraints
- A base rate assumption
- A catalyst assumption

Otherwise it sounds like optimism instead of calibration.

Step 1 — Define the Event Precisely

Not:

"My system succeeds."

Instead:

"By Dec 2026, X will exist and be used by Y in Z context."

For example (just structurally):

- "By end of 2026, I will have 5 paying stability audit clients at $X tier."
- "By Q4 2026, my coherence kernel will be deployed in at least one live operational workflow."
- "By 18 months, I will convert one public whitepaper into a paid advisory engagement."

You need concrete observables.

Step 2 — Name the Friction

For your system, friction might be:

- Market education cost
- Trust acquisition latency
- Capital runway constraints
- Signaling asymmetry
- Distribution channel weakness

When she says 40%, she's pricing friction.

You need to price yours.

Step 3 — Base Rate Honesty

Most independent technical frameworks:

- Do not become standards.
- Do not gain mass adoption.
- Do not generate durable income in year one.

That's not pessimism. That's base rate reality.

But you are not trying for mass adoption.

You're targeting:

- Niche
- Systems-aware operators
- High-friction, high-value engagements

That changes base rates upward.

Example Calibration (Illustrative Only)

If we were being disciplined:

- 70% chance you build a coherent, documented, versioned system.
- 50% chance you convert it into at least one paid engagement in 12 months.
- 30% chance it becomes a repeatable advisory revenue channel.
- 10–20% chance it becomes widely recognized beyond niche circles.

Notice:

Not cynical.

Not delusional.

Conditioned on execution.

Why This Matters

Percentages do two things:

- They force you to confront constraint.
- They prevent self-mythologizing.

You've said you don't want hype.

This is how you avoid it.

Now the real question:

What is the single, sharply-defined milestone you want to assign odds to?

Give me one.

We'll calibrate it cleanly.